Ensemble Methods

TP – Risk certificates for ensemble methods – Solutions

M2 Computer Science – MALIA

Guillaume Metzler
Institut de Communication (ICOM)
Université de Lyon, Université Lumière Lyon 2
Laboratoire ERIC UR 3083, Lyon, France

guillaume.metzler@univ-lyon2.fr

Abstract. In this practical session you will compute PAC-Bayesian risk certificates for random forests, Gibbs posteriors, boosted ensembles and linear classifiers on the datasets of the course, learn the weights of majority votes by minimizing these certificates, use them for model selection, and compare them with the classical hold-out certificate. The session relies on notebooks 1 to 5 and on the module `pbtools.py`. The work is organized in **four sessions of three hours**, each starting on a new page with its goals, the sections of the lecture notes to reread, a suggested timing and the results to hand in; parts are numbered by session (Part 2.3 is the third part of Session 2). The sessions build on each other, but the parts inside a session are independent after its first part and can be split among the members of a group. Parts marked ($\star$) are longer. All the numbers of these solutions are produced by the script `tp_solutions.py` (also available as the notebook `tp_pacbayes_solutions.ipynb`), whose functions are named after the parts (one notebook per session); each solution gives the pseudo-code of the computation, the results and their discussion. Unless stated otherwise, $\delta = 0.05$, splits are stratified 70/30 train/test splits (5 splits for Parts 1.3, 2.1, 3.3 and 3.4, 3 splits elsewhere), and forests have 100 trees trained on bootstrap samples of size $m/2$; your numbers will differ slightly with other seeds.

Session	Theme	Parts
1	Certifying a classifier and a forest	warm-up on the kl, selection bias, OOB certificates, trees and bootstrap size
2	Self-bounding learning with forests	learning the weights, exact kl minimization, strength vs diversity, model selection
3	Gibbs posteriors and hard cases	temperature of a Gibbs posterior, hold-out vs self-bounding, imbalance, boosting
4	Linear classifiers and synthesis	PBGD and Monte Carlo certificates, data-dependent priors, synthesis, report

Session 1 – Certifying one classifier and a random forest

Goal. By the end of the session you should be able to compute a test-set bound by hand and with the kl inversion, understand selection bias, and compute the first-order, tandem and C-bound certificates of a random forest from its out-of-bag samples.
To reread before the session. Sections 2, 5 and 7.2 of the notes; notebooks 1 and 3.
Timing (3 hours).

	Part 1.1	20 min
	Part 1.2	50 min
	Part 1.3	60 min
	Part 1.4	50 min

To hand in at the end of the session. Your notebook, with the table of Part 1.3 on your 10–15 datasets and the table of Part 1.4.

Part 1.1 – Setting up

- Download the datasets (`datasets.zip`) and the loading script `prepdata.py` of the first practical session, and the module `pbtools.py`. The function `data_recovery(name)` returns a pair `(X, y)` with labels in $\{0, 1\}$: convert them to $\{-1, +1\}$ and standardize the features.
- Read `pbtools.py`: every function is short. It provides the binary kl and its inversions (`kl_bin`, `kl_inv_upper`, `kl_inv_lower`), the classical bounds (`bound_seeger`, `bound_mcallester`, `bound_catoni`, `bound_lambda`), a random forest keeping track of its out-of-bag samples (`BaggedTrees` with `oob_losses`, `oob_tandem`, `oob_disagreement`), the majority-vote certificates (`fo_bound`, `tnd_bound`, `cbound_pb`), two optimizers (`optimize_fo`, `optimize_tnd`) and the gradient of the kl-form certificates (`certificate_and_grad`).
- Use **at least 10 datasets**, balanced and imbalanced, for instance `australian`, `balance`, `bupa`, `german`, `heart`, `iono`, `pima`, `sonar`, `wdbc`, `spambase` (balanced group) and `abalone8`, `pageblocks`, `satimage`, `segmentation`, `yeast3` (imbalanced group).

> **Good practice.** Fix all the random seeds, and write one function per experiment that returns a table: you will rerun them many times. Before computing a certificate, write down on paper *which examples* each voter has seen: this single question avoids most of the mistakes of this session.

Solution

All experiments use the same loading helper, which standardizes the features and maps the labels to $\{-1, +1\}$ (function `load`). The sizes range from 208 (`sonar`) to 6435 (`satimage`) examples, and the proportion of positive examples from 2% to 48%. Keep these sizes in mind: a PAC-Bayesian certificate is computed with the number of examples on which the voters are evaluated, and with only a few hundred examples it cannot be tight.

Part 1.2 – Warm-up: inverting the kl and certifying one classifier (Section 2)

1. Implement the upper inversion $\overline{\mathrm{kl}}^{-1}(\hat{q}, \varepsilon)$ by bisection (Algorithm 1 of the notes) and check it against `kl_inv_upper`. How many iterations are needed for a precision of 10^{-9}?
2. For $m \in \{100, 1000\}$, $\delta = 0.05$, $\varepsilon = \ln(1/\delta)/m$ and $\hat{q} \in \{0, 0.01, 0.05, 0.1, 0.3, 0.5\}$, compare three upper confidence bounds on the risk of a *fixed* classifier: Hoeffding $\hat{q} + \sqrt{\varepsilon/2}$, the kl inversion, and the refined Pinsker relaxation $\hat{q} + \sqrt{2\hat{q}\varepsilon} + 2\varepsilon$. When does the kl make a difference?
3. On `australian`, split the data in two halves. Learn a decision tree of depth 3 on the first half and compute the Hoeffding and kl test-set bounds of its risk on the second half.
4. Now learn trees of depths $1, \ldots, 10$ on the first half, keep the one with the smallest error on the second half, and compute its kl bound as if it had been fixed in advance. Why is this "certificate" not valid? Correct it.

Solution

1. The bisection keeps an interval $[a, b]$ that contains $\overline{\mathrm{kl}}^{-1}(\hat{q}, \varepsilon)$, i.e. such that $\mathrm{kl}(\hat{q}\|a) \leq \varepsilon < \mathrm{kl}(\hat{q}\|b)$; since $p \mapsto \mathrm{kl}(\hat{q}\|p)$ is increasing on $[\hat{q}, 1]$, the midpoint tells on which side the answer lies.

Algorithm 1: `kl_inv_bisection`$(\hat{q}, \varepsilon, \tau)$

$a \leftarrow \hat{q}$; $b \leftarrow 1$;
while $b - a > \tau$ **do**
  $c \leftarrow (a+b)/2$;
  if $\mathrm{kl}(\hat{q}\|c) > \varepsilon$ **then** $b \leftarrow c$ **else** $a \leftarrow c$;
return b ; ▷ *upper end, hence a valid (conservative) bound*

Python: function `kl_inv_bisection, part1` of `tp_solutions.py`.
The interval length is divided by 2 at each step, starting from at most 1: $\lceil \log_2 10^9 \rceil = 30$ iterations give a precision of 10^{-9}. The implementation agrees with `kl_inv_upper` to 10^{-6} on all the values of question 2.
2. Results ($\delta = 0.05$):

m	$\hat{q}$	Hoeffding	kl	refined Pinsker
100	0.00	0.122	0.030	0.060
100	0.01	0.132	0.056	0.094
100	0.05	0.172	0.122	0.165
100	0.10	0.222	0.189	0.237
100	0.30	0.422	0.418	0.494
100	0.50	0.622	0.621	0.733
1000	0.00	0.039	0.003	0.006
1000	0.01	0.049	0.020	0.024
1000	0.05	0.089	0.069	0.073
1000	0.10	0.139	0.125	0.130
1000	0.30	0.339	0.336	0.348
1000	0.50	0.539	0.539	0.561

The Hoeffding width $\sqrt{\ln(1/\delta)/(2m)}$ does not depend on $\hat{q}$ (0.122 for $m = 100$, 0.039 for $m = 1000$). The kl interval is much narrower when $\hat{q}$ is small: for $\hat{q} = 0$ it is $1 - e^{-\varepsilon} \approx \varepsilon$, i.e. a width in $1/m$ instead of $1/\sqrt{m}$ (0.003 instead of 0.039 for $m = 1000$). When $\hat{q}$ approaches $1/2$ the two bounds coincide, as predicted by Pinsker's inequality, which is tight around $1/2$. The refined Pinsker bound captures the fast rate for small $\hat{q}$, but it becomes looser than Hoeffding's for large $\hat{q}$ (0.494 against 0.422 at $\hat{q} = 0.3$, $m = 100$); the kl is always the tightest of the three. The kl matters for good classifiers, which is precisely the case of interest.
3–4.

Algorithm 2: Test-set bound, and the same bound after a selection among K candidates

Split S into S_1 (training) and S_2 (test), $n = |S_2|$;
for $k = 1, \dots, K$ **do**
  learn h_k on S_1; $\hat{r}_k \leftarrow$ error of h_k on S_2;
$k^\star \leftarrow \arg\min_k \hat{r}_k$;
naive certificate $\leftarrow \overline{\mathrm{kl}}^{-1}\big(\hat{r}_{k^\star}, \ln(1/\delta)/n\big)$; ▷ *invalid: S_2 was used to select*
corrected certificate $\leftarrow \overline{\mathrm{kl}}^{-1}\big(\hat{r}_{k^\star}, \ln(K/\delta)/n\big)$; ▷ *Occam / union bound*

Python: function `part1` of `tp_solutions.py`.

n	error (depth 3)	Hoeffding	kl	selected depth	its error	naive kl	kl with $\delta/10$
345	0.142	0.208	0.192	8	0.139	0.189	0.207

With $n = 345$ test examples, the depth-3 tree has a test error of 0.142; the Hoeffding bound gives 0.208 and the kl bound 0.192. The selected depth (8) has been chosen because its error on the second half is the smallest: that half is no longer independent of the selected classifier, and the test-set bound, which holds for a classifier *fixed in advance*, does not apply (selection

bias, Figure 18 of the notes). The correction is the Occam bound with a uniform prior on the 10 candidates, i.e. a union bound: replacing δ by $\delta/10$ increases the certificate from 0.189 to 0.207. The price, $\ln 10 \approx 2.3$ nats, is small; the mistake would be to pay nothing.

Part 1.3 – Certificates of a standard random forest (Sections 5 and 7.2)

For each dataset, repeat 5 times: split the data into 70% training / 30% test (stratified) and learn a `BaggedTrees` forest of 100 trees on the training set with bootstrap samples of size $m/2$.

1. Compute the test risk of the uniform majority vote, the test Gibbs risk and disagreement, and the three OOB certificates (first-order, tandem, C-bound) of the uniform posterior, with $\delta = 0.05$. Report also the sizes n_1 (smallest OOB set) and n_2 (smallest intersection of two OOB sets).
2. Check that the certificates are always above the test risk. Which bound is the tightest, on which datasets? Relate your answer to the disagreement of the trees (Proposition 5.7) and to n_1, n_2.
3. Compare the test risk of `BaggedTrees` with `sklearn.ensemble.RandomForestClassifier` (full bootstrap). How much accuracy do we lose by using bootstrap samples of size $m/2$?

Solution

Algorithm 3: OOB certificates of a forest (uniform posterior π)

for $t = 1, \dots, n$ **do**
  draw a bootstrap sample S_t of size $m/2$; learn tree h_t on S_t; $T_t \leftarrow S \setminus S_t$;
  $\widehat{L}_t \leftarrow$ error of h_t on T_t;
for t, t' **do**
  $\widehat{L}_{tt'} \leftarrow$ frequency of joint errors of $h_t, h_{t'}$ on $T_t \cap T_{t'}$; $\widehat{D}_{tt'} \leftarrow$ frequency of disagreements on $T_t \cap T_{t'}$;
$n_1 \leftarrow \min_t |T_t|$; $n_2 \leftarrow \min_{t,t'} |T_t \cap T_{t'}|$;
FO $\leftarrow 2\,\overline{\mathrm{kl}}^{-1}(\pi^\top \widehat{L}, \frac{\ln(2\sqrt{n_1}/\delta)}{n_1})$; TND $\leftarrow 4\,\overline{\mathrm{kl}}^{-1}(\pi^\top \widehat{\mathbf{L}}\pi, \frac{\ln(2\sqrt{n_2}/\delta)}{n_2})$;
$\overline{r} \leftarrow \overline{\mathrm{kl}}^{-1}(\pi^\top \widehat{L}, \cdot)$, $\underline{d} \leftarrow \underline{\mathrm{kl}}^{-1}(\pi^\top \widehat{\mathbf{D}}\pi, \cdot)$, each with $\delta/2$;
C-bound $\leftarrow 1 - \frac{(1-2\overline{r})^2}{1-2\underline{d}}$; ▷ *Equation (5.5) of the notes*

Python: function `forest_stats, certificates, part2_3` of `tp_solutions.py`.
Mean values over the 5 splits (RF: `RandomForestClassifier`; MV, Gibbs, d: uniform forest on the test set; FO, TND, C: OOB certificates of the uniform vote):

dataset	RF	MV	Gibbs	d	n_1	n_2	FO	TND	C-bound
australian	0.137	0.141	0.234	0.258	280	154	0.649	0.819	0.839
balance	0.114	0.097	0.171	0.205	253	138	0.523	0.664	0.731
bupa	0.260	0.254	0.401	0.425	136	70	1.000	1.000	1.000
german	0.245	0.242	0.346	0.364	408	226	0.864	1.000	0.968
heart	0.153	0.158	0.280	0.323	106	54	0.934	1.000	0.996
iono	0.074	0.077	0.158	0.190	139	72	0.560	0.790	0.792
pima	0.250	0.249	0.331	0.327	310	171	0.848	1.000	0.965
sonar	0.194	0.216	0.351	0.394	81	39	1.000	1.000	1.000
wdbc	0.056	0.061	0.093	0.100	231	124	0.324	0.476	0.528
spambase	0.051	0.056	0.124	0.156	1919	1119	0.306	0.289	0.380
abalone8	0.142	0.141	0.211	0.205	1741	1017	0.502	0.591	0.639
pageblocks	0.023	0.021	0.042	0.048	2283	1339	0.124	0.150	0.190
satimage	0.059	0.062	0.100	0.111	2695	1577	0.248	0.271	0.329
segmentation	0.024	0.027	0.070	0.093	958	551	0.210	0.221	0.310
yeast3	0.050	0.046	0.086	0.101	612	346	0.274	0.352	0.424

1–2. All certificates are above the test risk, as they must be (a violation would happen with probability at most 5% per split). Their quality depends above all on the number of OOB examples. On the small datasets (`bupa`, `sonar`, `heart`: $n_2 < 75$) all certificates are vacuous or nearly so: with 54 examples, $\ln(2\sqrt{n}/\delta)/n \approx 0.09$ even with KL $= 0$, and the factor 2 or 4 does the rest. On the larger datasets (`spambase`, `pageblocks`, `satimage`, `segmentation`) they become informative, e.g. 0.124 for `pageblocks` whose vote errs on 2.1% of the test points.
Which bound is the tightest? Proposition 5.7 says that, as *oracle* quantities, the C-bound and the tandem bound beat the first-order bound when $d \geq R(G)$. This condition holds on almost all datasets (compare the columns Gibbs and d), but the certificates tell a different story: the first-order certificate is the smallest on every dataset except `spambase` (on `bupa` and `sonar` all three are vacuous). The reason is statistical: the tandem bound is estimated on $n_2 \approx 0.55\, n_1$ examples and multiplied by 4, and the C-bound needs two kl inversions. Only when the sample is large and the trees diverse enough (`spambase`: $d = 0.156 > R = 0.124$, $n_2 = 1119$) does the tandem certificate win. The best oracle bound is not necessarily the best certificate.
3. The two forests have very close test risks (e.g. 0.051 vs 0.056 on `spambase`, 0.137 vs 0.141 on `australian`); on some datasets `BaggedTrees` is even slightly better. Halving the bootstrap samples costs little accuracy, while it raises the OOB proportions from $e^{-1} \approx 37\%$ to $e^{-1/2} \approx 61\%$ (and from 14% to 37% for pairs), which makes the certificates much tighter. Part 1.4 studies this trade-off.

Part 1.4 – Number of trees and size of the bootstrap samples (Section 7.2)

On `spambase` (3 splits), learn forests with $n \in \{10, 50, 100, 200\}$ trees and bootstrap samples of size $k = f\, m$ with $f \in \{0.3, 0.5, 0.8, 1\}$.

1. Before running anything, predict how n_1 and n_2 depend on f (compute $(1 - 1/m)^{fm}$ and its square), and whether they depend on n.

2. Report the test risk of the uniform vote, n_1, n_2 and the FO and TND certificates. Which configuration gives the best vote? the best certificate? Is there a configuration that is good for both?

Solution

1. An example is out of a bootstrap sample of size fm with probability $(1 - \frac{1}{m})^{fm} \approx e^{-f}$, and out of two independent samples with probability $\approx e^{-2f}$. With $m = 3217$ training examples, $n_1 \approx e^{-f} m$ gives $2383, 1951, 1446, 1183$ and $n_2 \approx e^{-2f} m$ gives $1765, 1183, 650, 435$ for $f = 0.3, 0.5, 0.8, 1$; the minima over trees and pairs are a little smaller. The expected sizes do not depend on n, but the minima decrease slowly with n (a minimum over more

trees or pairs).

Algorithm 4: Grid over the number of trees and the bootstrap size

```
for f ∈ {0.3, 0.5, 0.8, 1}, n ∈ {10, 50, 100, 200}, split s = 1, 2, 3 do
    learn BaggedTrees(n, f) on the training part; compute the OOB statistics
      (Part 1.3);
    record test risk of the uniform vote, n_1, n_2, FO and TND certificates;
average over the splits;
```

Python: function `partA` of `tp_solutions.py`.

f	n	test risk	n_1	n_2	FO cert.	TND cert.
0.3	10	0.078	2370	1734	0.334	0.331
0.3	50	0.058	2361	1708	0.326	0.292
0.3	100	0.057	2358	1706	0.327	0.286
0.3	200	0.057	2358	1706	0.328	0.284
0.5	10	0.071	1931	1146	0.310	0.323
0.5	50	0.054	1922	1128	0.307	0.293
0.5	100	0.055	1918	1117	0.308	0.291
0.5	200	0.054	1918	1109	0.308	0.287
0.8	10	0.065	1422	615	0.300	0.354
0.8	50	0.053	1408	586	0.296	0.322
0.8	100	0.049	1406	584	0.299	0.321
0.8	200	0.049	1402	577	0.300	0.319
1.0	10	0.067	1141	392	0.291	0.393
1.0	50	0.050	1136	380	0.299	0.362
1.0	100	0.048	1136	371	0.296	0.355
1.0	200	0.050	1135	370	0.297	0.356

2. The measured n_1, n_2 match the prediction. *Number of trees:* going from 10 to 50 trees improves the vote ($0.078 \to 0.058$ for $f = 0.3$) and the tandem certificate ($0.331 \to 0.292$), because the joint errors of more trees average out; beyond 100 trees nothing changes. *Bootstrap size:* larger bootstrap samples give better trees and a slightly better vote (0.057 for $f = 0.3$, 0.048 for $f = 1$ with 100 trees) and a slightly smaller first-order certificate, but they shrink the pairwise OOB sets from 1706 to 371 examples, and the tandem certificate degrades from 0.286 to 0.355. There is no free lunch: the best vote ($f = 1$) and the best certificate ($f = 0.3$, 200 trees: 0.284) are obtained for different configurations. $f = 0.5$ with 100 trees is a reasonable compromise (0.055 / 0.291), which is why it is the default of the course. Note that choosing the configuration *after* seeing the certificates would require a union bound over the 16 configurations ($\ln 16 \approx 2.8$ nats).

Session 2 – Self-bounding learning with random forests

Goal. By the end of the session you should be able to learn the weights of a forest by minimizing a certificate, compare relaxed and exact objectives, relate strength and diversity to the certificates, and select a model with certificates instead of cross-validation.
To reread before the session. Sections 5, 6 and 7.2 of the notes; notebook 4.
Timing (3 hours). Part 2.1 60 min
Part 2.2 45 min
Part 2.3 30 min
Part 2.4 45 min
To hand in at the end of the session. Your notebook, with the table of Part 2.1 with means and standard deviations, and the model-selection table of Part 2.4.

Part 2.1 – Learning the weights (Sections 6 and 7.2)

1. On each dataset, compute the posteriors minimizing the first-order bound (`optimize_fo`) and the tandem bound (`optimize_tnd`). Report the test risk of the corresponding votes, their certificates and the *effective number of trees* $1/\sum_t \rho_t^2$.
2. Produce a table with one row per dataset and, for each posterior (uniform, FO, TND), the mean and standard deviation over the 5 splits of the test risk and of the certificate. For the uniform posterior, report the best of the three certificates: what correction does this choice require?
3. Which posterior gives the best vote? the best certificate? Are the conclusions of notebook 4 confirmed?

Solution

Algorithm 5: Learning the weights of the forest (FO: alternating minimization; TND: L-BFGS on logits + closed-form λ)

Input: OOB statistics $\widehat{L}$, $\widehat{\mathbf{L}}$, n_1, n_2; prior π uniform

$\rho_{\mathrm{FO}} \leftarrow$ Algorithm 3 of the notes: repeat $\rho \propto \pi e^{-\lambda n_1 \widehat{L}}$, $\lambda \leftarrow \lambda^\star(\rho)$;

$\rho_{\mathrm{TND}} \leftarrow$ Algorithm 4 of the notes: repeat $\theta \leftarrow \arg\min_\theta \mathcal{B}_{\mathrm{TND}}(\mathrm{softmax}(\theta), \lambda)$, $\lambda \leftarrow \lambda^\star$;

for $\rho \in \{\pi, \rho_{\mathrm{FO}}, \rho_{\mathrm{TND}}\}$ **do**

  test risk of B_ρ; effective number $1/\sum_t \rho_t^2$;

certificates: FO of ρ_{FO}, TND of ρ_{TND}, and $\min(\mathrm{FO}, \mathrm{TND}, \mathrm{C})$ of π with $\delta/3$ each;

Python: function `part2_3` of `tp_solutions.py`.
Means (standard deviations) over the 5 splits; "eff." is the effective number of trees. For the uniform vote, the certificate is the minimum of the FO, TND and C-bound certificates computed with $\delta/3$ each (union bound, Proposition 6.3), since the best bound is chosen after seeing them.

dataset	uniform test risk	uniform best cert.	FO-optimized test risk	FO-optimized FO cert.	FO-optimized eff.	TND-optimized test risk	TND-optimized TND cert.	TND-optimized eff.
australian	0.141 (0.011)	0.666 (0.006)	0.170 (0.029)	0.509 (0.019)	3.2	0.138 (0.013)	0.772 (0.013)	58.3
balance	0.097 (0.016)	0.540 (0.014)	0.137 (0.025)	0.442 (0.023)	5.6	0.101 (0.020)	0.633 (0.029)	71.8
bupa	0.254 (0.038)	1.000 (0.000)	0.367 (0.066)	0.985 (0.019)	5.0	0.283 (0.049)	1.000 (0.000)	66.8
german	0.242 (0.010)	0.879 (0.011)	0.297 (0.035)	0.761 (0.027)	2.3	0.248 (0.014)	0.998 (0.003)	51.8
heart	0.158 (0.044)	0.964 (0.025)	0.212 (0.040)	0.797 (0.027)	8.0	0.165 (0.040)	1.000 (0.000)	74.4
iono	0.077 (0.026)	0.584 (0.016)	0.079 (0.014)	0.440 (0.020)	7.5	0.075 (0.028)	0.739 (0.022)	76.6
pima	0.249 (0.021)	0.864 (0.023)	0.268 (0.028)	0.765 (0.016)	6.0	0.247 (0.023)	0.999 (0.001)	57.6
sonar	0.216 (0.022)	1.000 (0.000)	0.305 (0.027)	0.897 (0.049)	6.6	0.206 (0.010)	1.000 (0.000)	74.3
wdbc	0.061 (0.014)	0.340 (0.014)	0.070 (0.018)	0.258 (0.024)	10.3	0.060 (0.015)	0.438 (0.018)	72.5
spambase	0.056 (0.005)	0.298 (0.004)	0.096 (0.015)	0.263 (0.007)	1.9	0.054 (0.003)	0.272 (0.005)	46.0
abalone8	0.141 (0.003)	0.507 (0.002)	0.178 (0.030)	0.472 (0.005)	5.3	0.143 (0.002)	0.572 (0.003)	51.3
pageblocks	0.021 (0.002)	0.127 (0.001)	0.034 (0.006)	0.105 (0.002)	3.2	0.022 (0.002)	0.140 (0.004)	46.2
satimage	0.062 (0.002)	0.252 (0.004)	0.094 (0.004)	0.227 (0.005)	2.1	0.061 (0.004)	0.258 (0.006)	47.5
segmentation	0.027 (0.010)	0.216 (0.005)	0.049 (0.013)	0.152 (0.010)	1.6	0.025 (0.009)	0.199 (0.004)	47.8
yeast3	0.046 (0.009)	0.282 (0.008)	0.053 (0.008)	0.220 (0.005)	6.3	0.048 (0.006)	0.325 (0.011)	56.0
mean	0.123	0.568	0.161	0.486		0.125	0.623	

Best vote. The TND-optimized vote is as good as the uniform one (mean 0.125 vs 0.123, differences within one standard deviation on every dataset). The FO-optimized vote is worse on all 15 datasets (only marginally on `iono`; mean 0.161), sometimes by a lot (`bupa`: 0.254 → 0.367, `sonar`: 0.216 → 0.305). The effective number of trees explains why: the FO posterior keeps between 2 and 10 trees, the TND posterior about 50 to 75. Minimizing a first-order bound selects the individually best trees, which are correlated, and destroys the diversity that makes the vote work.
Best certificate. Paradoxically, the smallest certificate is almost always the FO certificate of the FO-optimized posterior (mean 0.486), because it is estimated on more examples and it minimizes precisely this number. But it certifies a worse vote. The TND certificate of the TND-optimized posterior is smaller than (or equal to, when both are vacuous) the TND certificate of the uniform vote on every dataset (e.g. 0.289 → 0.272 on `spambase`), and it becomes the best compromise when the data are large enough. This reproduces the conclusions of notebook 4 and of Section 7.2: a certificate that ignores the correlations between voters should not be used as a learning objective for a vote.

Part 2.2 – Minimizing the kl certificate directly (Section 6.6)

The optimizers of Part 2.1 minimize a *relaxation* of the certificate (PAC-Bayes-λ). Since the certificate is valid for any posterior, we can also minimize the kl-form certificate itself.

1. Using `certificate_and_grad` (value and gradient of $4\,\overline{\mathrm{kl}}^{-1}(\rho^\top \widehat{\mathbf{L}}\rho, \varepsilon(\rho))$ with respect to the softmax logits, Proposition 6.6), implement Adam (300 iterations, step 0.05, starting from the uniform posterior).

2. On `spambase`, `satimage`, `wdbc` and `pageblocks`, compare the uniform posterior, the TND-optimized posterior of Part 2.1 and the posterior found by Adam: certificate, test risk, effective number of trees. Plot the certificate along the iterations.

3. Does the choice of the optimizer, of the step size or of the number of iterations require a union bound? Why?

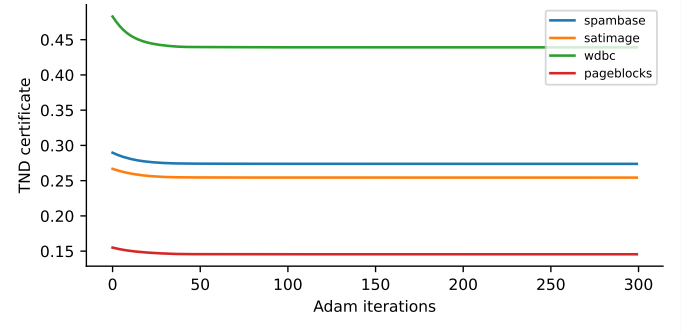
Solution

Algorithm 6: Adam on the kl-form tandem certificate

$\theta \leftarrow \ln \pi$; $u \leftarrow 0$; $v \leftarrow 0$;
for $k = 1, \dots, K$ **do**
  $(\mathcal{B}, g) \leftarrow$ `certificate_and_grad`(θ) ; ▷ $g = \rho \odot (g_\rho - \rho^\top g_\rho)$, g_ρ *from the kl derivatives*
  $u \leftarrow 0.9u + 0.1g$; $v \leftarrow 0.999v + 0.001g^2$;
  $\theta \leftarrow \theta - \eta \frac{u/(1-0.9^k)}{\sqrt{v/(1-0.999^k)}+10^{-8}}$;
return $\rho = \mathrm{softmax}(\theta)$ and its certificate;

Python: function `adam_certificate, partB` of `tp_solutions.py`.

dataset	uniform cert.	uniform test	TND-λ (Part 2.1) cert.	TND-λ test	TND-λ eff.	Adam on the kl certificate cert.	Adam test	Adam eff.
spambase	0.291	0.055	0.274	0.053	47	0.273	0.053	41
satimage	0.268	0.063	0.255	0.064	44	0.255	0.064	39
wdbc	0.470	0.060	0.432	0.058	72	0.424	0.058	42
pageblocks	0.151	0.021	0.140	0.021	46	0.140	0.021	36



2. Both optimizers improve the certificate of the uniform forest by a few points (0.291 → 0.273 on `spambase`, 0.470 → 0.424 on `wdbc`) without changing the test risk of the vote. Minimizing the exact kl certificate is marginally better than minimizing its PAC-Bayes-λ relaxation (at most 0.008 on `wdbc`), which confirms that the relaxation is tight near the optimum; Adam keeps slightly fewer effective trees (36–42 instead of 44–72). The curves show that most of the gain is obtained in the first 50 iterations.
3. No union bound is needed: the optimizer, its hyper-parameters and its stopping time only determine *which* posterior is returned, and the certificate is valid for every posterior simultaneously (Theorem 6.2). One may therefore try several optimizers and keep the best certificate, provided the prior, δ and the form of the bound are unchanged.

Part 2.3 – Strength versus diversity (Section 5)

1. On `spambase` and `german`, learn forests with `max_features` $\in \{1, \sqrt{d}, d/2, d\}$ (3 splits). Report the test Gibbs risk, the test disagreement, the test risk of the vote and the OOB tandem and C-bound certificates.
2. Describe how strength and diversity evolve. Which value gives the best vote? Do the certificates follow the risk of the vote?

Solution

Algorithm 7: Strength–diversity study

for *max_features* $\in \{1, \sqrt{d}, d/2, d\}$, *split* $s = 1, 2, 3$ **do**
  learn the forest; on the test sample compute $R(G_\pi)$, $d_\pi = 2(R(G_\pi) - e_\pi)$ and $R(B_\pi)$;
  compute the OOB TND and C-bound certificates (Part 1.3);

Python: function `part5` of `tp_solutions.py`.

dataset	`max_features`	Gibbs	d	vote	TND cert.	C-bound cert.
spambase	1	0.142	0.192	0.048	0.307	0.413
spambase	$\sqrt{d}$	0.123	0.158	0.055	0.291	0.382
spambase	$d/2$	0.116	0.136	0.060	0.302	0.377
spambase	d	0.114	0.128	0.064	0.309	0.379
german	1	0.362	0.376	0.264	1.000	0.981
german	$\sqrt{d}$	0.346	0.363	0.244	1.000	0.969
german	$d/2$	0.335	0.349	0.234	1.000	0.961
german	d	0.333	0.343	0.242	1.000	0.959

Trying more features per split makes each tree stronger (the Gibbs risk decreases, from 0.142 to 0.114 on `spambase`) but the trees more similar (the disagreement decreases from 0.192 to 0.128). The vote is best with a single feature on `spambase` (0.048) and with $d/2$ features on `german` (0.234): the optimum is a trade-off, as predicted by the C-bound (Figure 35 of the notes). On `spambase` the tandem certificate is minimal at $\sqrt{d}$ (0.291) and the C-bound certificate decreases and then stabilizes: the certificates see the trade-off but not its exact optimum, because their estimation error varies with the configuration. On `german` (700 training examples) the tandem certificate is vacuous and the C-bound certificate hardly informative: with few examples, certificates cannot guide such a fine choice.

Part 2.4 – Self-bounding model selection (Section 6)

1. On `spambase`, `pima`, `wdbc` and `german`, learn $K = 5$ forests with `max_depth` $\in \{1, 2, 4, 8, \text{None}\}$, minimize the tandem bound of each of them and compute its certificate with δ/K. Select the depth with the smallest certificate.
2. Compare with the depth selected by 5-fold cross-validation of the uniform forest on the training set, and with the best depth in hindsight (smallest test risk). Report the test risks of the selected models.
3. Why must the certificates be computed with δ/K? When does this selection rule fail, and what should be done then?

Solution

Algorithm 8: Model selection with certificates versus cross-validation

for $k = 1, \ldots, K$ *(depth* D_k*)* **do**
  forest F_k on the training set; $\rho_k \leftarrow$ `optimize_tnd` with δ/K; $c_k \leftarrow$ TND certificate with δ/K;
  $\text{cv}_k \leftarrow$ 5-fold cross-validation error of the uniform forest of depth D_k;
$k_{\text{cert}} \leftarrow \arg\min_k c_k$ (certified at level $1 - \delta$); $k_{\text{CV}} \leftarrow \arg\min_k \text{cv}_k$ (no certificate);

Python: function `part6` of `tp_solutions.py`.
Selected depths on 3 splits, mean test risk of the selected forest, and mean certificate of the model selected by the certificate:

dataset	depth (cert.)	test risk	depth (CV)	test risk	best	certificate
spambase	None, None, None	0.052	None, None, None	0.052	0.052	0.286
pima	1, 1, 1	0.254	4, None, 4	0.238	0.225	1.000
wdbc	8, 8, 8	0.058	4, 4, 8	0.058	0.055	0.489
german	1, 1, 1	0.294	None, None, None	0.247	0.240	1.000

Why δ/K. Keeping the smallest of K certificates is a choice made with the data; the certificate of the selected model is valid only if all K certificates hold simultaneously, which the union bound guarantees (Proposition 6.3). The price is $\ln 5 \approx 1.6$ nats, negligible here.
When it works. On `spambase` and `wdbc`, where the certificates are informative (0.29 and 0.49), the certificate selects deep trees and reaches the same test risk as cross-validation, without any validation fold and with a guarantee on the selected model.
When it fails. On `pima` and `german`, all the certificates equal 1: they are vacuous and carry no information, and the argmin returns the first candidate (depth 1), which is a poor choice (0.254 vs 0.238 for cross-validation on `pima`, 0.294 vs 0.247 on `german`). A selection rule based on certificates is meaningful only when the certificates are non-vacuous: with small samples, one should fall back on cross-validation (and certify the selected model on a held-out sample, Part 3.2), or use a first-order or data-dependent-prior certificate, which is less loose on small data.

Session 3 – Gibbs posteriors, hold-out, imbalance and boosting

Goal. By the end of the session you should be able to build voters so that the prior is legitimate, compare self-bounding and hold-out certificates, certify the balanced error, and understand why boosted votes are hard to certify.
To reread before the session. Sections 4, 6 and 7.3 of the notes; notebooks 2 and 4.
Timing (3 hours).

	Part 3.1	50 min
	Part 3.2	35 min
	Part 3.3	50 min
	Part 3.4	45 min

To hand in at the end of the session. Your notebook, with the figure of Part 3.1 and the tables of Parts 3.2 to 3.4.

Part 3.1 – The Gibbs posterior and its temperature (Section 4)

We now build the voters ourselves, so that the prior can be fixed before seeing the data used in the bound.

1. On `wdbc` and `australian`, keep 20% of the data to build the voters: all the decision stumps $x \mapsto s\,\mathrm{sign}(x_j - \theta)$, $s \in \{\pm 1\}$, whose thresholds θ are the quantiles $10, 25, 50, 75, 90\%$ of feature j on this part. Split the remaining data into a sample S (60%) used by the bound and a test sample.

2. With the uniform prior, compute the Gibbs posterior $Q_\lambda \propto e^{-\lambda m \widehat{R}_S(h)}$ for a grid of 60 values of λ between 10^{-3} and $10^{1.5}$, and plot its empirical Gibbs risk, $\mathrm{KL}(Q_\lambda \| P)/m$ and Seeger's bound as functions of λ.

3. Report the best certificate, the corresponding test Gibbs risk and test risk of the vote, and compare with the Occam bound of the stump with the smallest empirical risk. Why is choosing λ on the grid free?

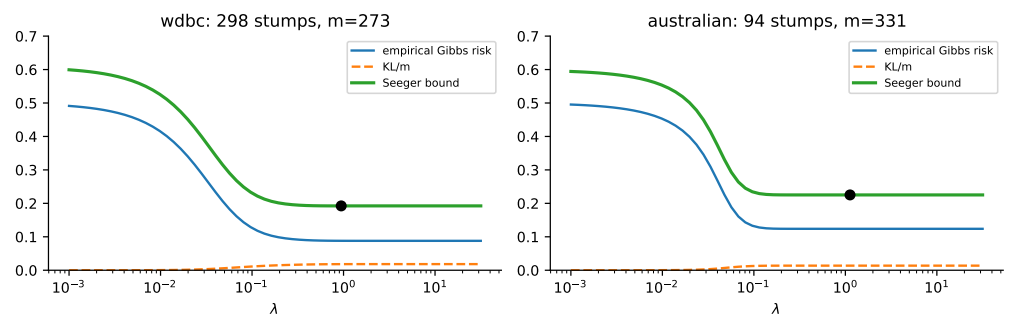
> **Numerical tip.** Compute the Gibbs posterior with a stable softmax, $Q_\lambda = \mathrm{softmax}(\ln P - \lambda m \widehat{R}_S)$, which subtracts the maximum before exponentiating; the naive formula underflows as soon as λm is large.

Solution

Algorithm 9: Gibbs posterior on stumps and choice of the temperature

split the data into S_{prior} (20%), S and a test sample;
$\mathcal{H} \leftarrow$ stumps with thresholds at the quantiles of each feature on S_{prior}; $P \leftarrow$ uniform on $\mathcal{H}$;
$\widehat{R}_S(h) \leftarrow$ error of each stump on S ($m = |S|$);
for λ *in a geometric grid* **do**
  $Q_\lambda \leftarrow \mathrm{softmax}(\ln P - \lambda m \widehat{R}_S)$;
  $B(\lambda) \leftarrow \overline{\mathrm{kl}}^{-1}(Q_\lambda^\top \widehat{R}_S, \frac{\mathrm{KL}(Q_\lambda \| P) + \ln(2\sqrt{m}/\delta)}{m})$;
return $\lambda^\star = \arg\min B(\lambda)$, $Q_{\lambda^\star}$, $B(\lambda^\star)$; ▷ *free:* λ *indexes posteriors*

Python: function `stumps, stump_votes, part4` of `tp_solutions.py`.



				best Gibbs posterior					ERM stump (Occam)	
dataset	n	m	λ	$\widehat{R}_S(G)$	KL	Seeger	test Gibbs	test vote	certificate	test
wdbc	298	273	0.943	0.088	4.98	0.192	0.104	0.104	0.196	0.104
australian	94	331	1.124	0.124	4.54	0.225	0.158	0.158	0.225	0.158

Shape of the curves. For small λ the posterior is the prior; since every stump comes with its opposite, the empirical Gibbs risk is exactly 0.5 and the bound is vacuous. When λ grows the empirical risk drops quickly while KL/m grows slowly, up to $\ln(n)/m$, and the bound reaches a minimum. Beyond this point the KL keeps increasing while the empirical risk no longer decreases.
Numbers. The best certificates are 0.192 (`wdbc`) and 0.225 (`australian`), for test Gibbs risks of 0.104 and 0.158. They are obtained for $\lambda \approx 1$, where the posterior is already concentrated on a few stumps (KL = 4.98 and 4.54 nats, close to $\ln n = 5.70$ and 4.54): on these problems a single feature is very informative, and the best Gibbs posterior is nearly the ERM stump, so its certificate is essentially the Occam bound (0.196 and 0.225) and its vote is essentially that stump. The Gibbs posterior does better than Occam when several voters are about equally good, as with the 300 stumps of `breast cancer` in Section 4 of the notes.
Why is the grid free? λ only indexes posteriors: Seeger's bound holds simultaneously for all Q, hence for all Q_λ, whatever the way λ is chosen (Theorem 6.2). No union bound is needed, contrary to the parameter C of Catoni's bound (Section 6.3).

Part 3.2 – Hold-out certificate versus self-bounding certificate (Section 6.4)

1. *Self-bounding:* learn a forest on the whole training set, minimize its tandem bound, and report its TND certificate. *Hold-out:* learn the same forest on 70% of the training set and compute the kl test-set bound of its (uniform) vote on the remaining 30%. Compare the test risks and the certificates on `spambase`, `satimage`, `pageblocks`, `abalone8`, `wdbc`, `german`.
2. Which approach gives the better vote? the better certificate? Explain the gap between the two certificates.
3. In which situation would you nevertheless recommend the self-bounding approach?

Solution

Algorithm 10: Hold-out versus self-bounding

; ▷ *self-bounding: all the training data*
forest on S; $\rho \leftarrow$ `optimize_tnd`; certificate $\leftarrow$ TND certificate of ρ (OOB);
; ▷ *hold-out*
split $S = S_1 \cup S_2$ (70/30); forest on S_1; $\hat{r} \leftarrow$ error of its vote on S_2;
certificate $\leftarrow \overline{\mathrm{kl}}^{-1}(\hat{r}, \ln(1/\delta)/|S_2|)$;

Python: function `partC` of `tp_solutions.py`.

dataset	m	self-bounding test risk	self-bounding certificate	hold-out test risk	hold-out certificate
spambase	3217	0.053	0.274	0.054	0.071
satimage	4504	0.064	0.255	0.066	0.078
pageblocks	3831	0.021	0.140	0.025	0.038
abalone8	2923	0.143	0.571	0.142	0.169
wdbc	398	0.058	0.432	0.062	0.105
german	700	0.248	0.998	0.247	0.340

2. The votes are practically equally accurate: withholding 30% of the training set costs at most 0.004 of test risk here (`pageblocks`: 0.021 vs 0.025), because forests saturate quickly. The certificates, however, differ by a factor 3 to 5: 0.071 against 0.274 on `spambase`, 0.038 against 0.140 on `pageblocks`, 0.340 against 0.998 on `german`. The hold-out certifies the vote *itself* with a single kl inversion and no KL term, whereas the self-bounding certificate goes through the Gibbs-to-vote relation (factor 4) and uses the smaller pairwise OOB samples. This reproduces, on the datasets of the course, the conclusions of Section 6.4 of the notes (Figure 42).
3. Self-bounding is preferable when the certificate is also used to *learn* or to *select* (Parts 2.1 to 2.4: it then comes for free, with a union bound in $\ln K$), when data are too scarce to set a test sample aside, or for Gibbs classifiers and stochastic predictors, for which the Gibbs-to-vote factor disappears (Part 4.1). For reporting a guarantee on a fixed forest with enough data, the hold-out is hard to beat.

Part 3.3 – Imbalanced datasets and the balanced error (Section 6)

The bounds of this course control the 0-1 risk. On imbalanced datasets a trivial classifier predicting the majority class has a small 0-1 risk.

1. On the imbalanced datasets, compare the certificates of the uniform forest with the risk of the trivial classifier. Are the certificates informative?
2. Obtain a certificate on the *balanced* error $\frac{1}{2}\big(\mathbb{P}(h(x)\neq y\,|\,y=+1)+\mathbb{P}(h(x)\neq y\,|\,y=-1)\big)$: apply the OOB bounds separately to the training examples of each class (with n_+ and n_- OOB examples) and use a union bound. Implement it and comment.

Solution

Construction. Conditionally on $y=+1$, the examples of the positive class are i.i.d. from $\mathcal{D}_{|y=+1}$, and the OOB construction applies to them: restricting the OOB masks to the positive training examples gives OOB losses and tandem losses whose expectations are the class-conditional error rates. We compute the FO and TND certificates on each class with $\delta/4$ (2 classes $\times$ 2 bounds), keep the smallest on each class, and average them: with probability $\geq 1-\delta$ both class-conditional certificates hold, hence their average bounds the balanced error.

Algorithm 11: Certificate on the balanced error

for $c\in\{+1,-1\}$ **do**
  $I_c\leftarrow\{i : y_i=c\}$; restrict every OOB set to I_c: $T_t^c\leftarrow T_t\cap I_c$;
  compute $\widehat{L}^c$, $\widehat{\mathbf{L}}^c$, n_1^c, n_2^c on these restricted sets;
  $\mathcal{B}_c\leftarrow\min\big(\mathrm{FO}(\delta/4),\mathrm{TND}(\delta/4)\big)$; ▷ *2 classes × 2 bounds*
return $\frac{1}{2}(\mathcal{B}_{+1}+\mathcal{B}_{-1})$; ▷ *valid with probability* $\geq 1-\delta$

Python: function `class_restricted, part7` of `tp_solutions.py`.

	trivial	vote	0-1 cert.	positives: error	cert.	balanced error	balanced cert.
abalone8	0.136	0.141	0.505	0.931	1.000	0.473	0.630
pageblocks	0.102	0.021	0.125	0.110	0.680	0.060	0.380
satimage	0.097	0.062	0.250	0.537	1.000	0.274	0.565
segmentation	0.143	0.027	0.214	0.143	0.837	0.076	0.492
yeast3	0.110	0.046	0.279	0.265	1.000	0.142	0.594

1. The trivial classifier has a 0-1 risk equal to the proportion of positives (0.10 to 0.14). The 0-1 certificates of the forest are larger than this value on every dataset, even on `pageblocks` (0.125 vs 0.102) where the forest is five times better than the trivial classifier on the test set. In other words, the 0-1 certificates cannot even guarantee that the forest is better than predicting the majority class.

2. The balanced error reveals what the 0-1 risk hides: on `abalone8` the vote misses 93% of the positive examples and its balanced error (0.47) is that of a random guess, although its 0-1 risk is 0.141. The balanced certificates are loose, because the positive class contains only a few hundred examples (0.68 on `pageblocks`, vacuous on three datasets), but they are honest: they certify what matters for an imbalanced problem, and they show that informative guarantees on the minority class require many minority examples. A natural improvement is to rebalance the bootstrap samples so that the trees see more positives.

Part 3.4 – Boosting (Section 7.3)

AdaBoost also outputs a weighted majority vote. To obtain a certificate, the voters must be evaluated on examples independent of them.

1. Split the training set into two halves S_1, S_2. Run `AdaBoostClassifier` with decision stumps (100 rounds) on S_1, and consider the learned stumps as the voters, with the uniform prior.
2. Compute on S_2 the FO and TND certificates of the posterior $\rho_t \propto \alpha_t$ given by AdaBoost (with $\delta/2$ each), and of the posteriors minimizing the two bounds. Compare the test risks with those of AdaBoost trained on the whole training set.
3. Why are the certificates of the AdaBoost posterior loose, although AdaBoost is accurate? *Hint:* compute the Gibbs risk of the stumps on S_2 and the KL of the AdaBoost posterior.

Solution

Algorithm 12: Certifying AdaBoost with an independent half

split the training set into S_1, S_2 ($n = |S_2|$);
run AdaBoost on S_1: stumps $h_1, \dots, h_T$ and weights α_t; $\pi \leftarrow$ uniform; $\rho_{\text{Ada}} \leftarrow \alpha / \sum_t \alpha_t$;
on S_2: $\widehat{L}_t \leftarrow$ error of h_t; $\widehat{L}_{tt'} \leftarrow$ joint errors of $h_t, h_{t'}$;
certificates of ρ_{Ada}: FO and TND with $\delta/2$ each;
$\rho_{\text{FO}}, \rho_{\text{TND}} \leftarrow$ `optimize_fo`, `optimize_tnd` on these statistics; their certificates;

Python: function `part8` of `tp_solutions.py`.

dataset	AdaBoost (all data)	AdaBoost (S_1)	AdaBoost posterior on S_2 Gibbs	KL	FO cert.	TND cert.	FO-optimized test	FO-optimized FO cert.	TND-optimized test	TND-optimized TND cert.
australian	0.148	0.150	0.430	0.30	1.000	1.000	0.147	0.541	0.147	0.908
german	0.245	0.242	0.453	0.39	1.000	1.000	0.301	0.782	0.287	1.000
pima	0.257	0.263	0.443	0.29	1.000	1.000	0.268	0.789	0.247	1.000
spambase	0.064	0.065	0.399	0.29	0.900	0.821	0.204	0.526	0.104	0.561
wdbc	0.043	0.043	0.265	0.03	0.783	0.709	0.103	0.332	0.063	0.465

2. Training on half of the data costs AdaBoost almost nothing (0.148 → 0.150 on `australian`).

The certificates of its posterior, however, are vacuous on three datasets and loose on the other two (0.71 on `wdbc`). Reweighting the stumps by minimizing the bounds improves the certificates markedly (TND certificate 0.54 on `australian`, 0.33 on `wdbc`), but mostly at the expense of the vote: the TND-optimized vote errs on 20% of `spambase` instead of 6.5%, and on 10% of `wdbc` instead of 4.3%. The optimized posteriors concentrate on the few stumps that are good on their own, and lose the combination built by AdaBoost.

3. The explanation lies in the nature of boosting. Its voters are *weak* on purpose: the Gibbs risk of the stumps on S_2 is 0.40–0.45 on four datasets out of five (0.27 on `wdbc`). AdaBoost makes them accurate *together*, by putting them in a precise combination with large margins, not by making each of them good. The first-order bound is then $2 \times 0.43 > 0.8$, and the tandem bound, which only uses the joint errors of pairs, cannot see a combination that relies on many voters at once. The KL of the AdaBoost posterior is small (0.03 to 0.4 nats), so the complexity is not the problem: the looseness comes from the Gibbs-to-vote relation. This is why boosted votes are better analysed with margin bounds (Section 5.9), or certified with a hold-out sample (Part 3.2).

Session 4 – Linear classifiers, data-dependent priors and synthesis

Goal. By the end of the session you should be able to implement PBGD, certify a Gaussian posterior exactly and by Monte Carlo, learn a prior on part of the data, and put all the results of the four sessions together.

To reread before the session. Sections 4.2, 4.3, 6.4 and 7.4 of the notes; notebook 5.

Timing (3 hours).

Part 4.1	75 min
Part 4.2	50 min
Part 4.3	30 min
Part 4.4	25 min

To hand in at the end of the session. Your notebook, with the tables of Parts 4.1 and 4.2, the decision table of Part 4.3, and the outline of the final report.

Part 4.1 – (⋆) PAC-Bayesian learning of linear classifiers and Monte Carlo certificates (Sections 4.3 and 7.4)

Consider linear classifiers $h_{\mathbf{v}}(x) = \operatorname{sign}\langle \mathbf{v}, \tilde{x}\rangle$ on the standardized features augmented with a constant 1 ($\tilde{x} = (x, 1)$), the prior $P = \mathcal{N}(\mathbf{0}, I)$ and the posteriors $Q_{\mathbf{w}} = \mathcal{N}(\mathbf{w}, I)$.

1. Implement the exact empirical Gibbs risk $\widehat{R}_S(G_{Q_{\mathbf{w}}}) = \frac{1}{m}\sum_i \Phi(-y_i\langle \mathbf{w}, \tilde{x}_i\rangle / \|\tilde{x}_i\|)$ and the PBGD objective $C\sum_i \Phi(-y_i\langle \mathbf{w}, \tilde{x}_i\rangle/\|\tilde{x}_i\|) + \frac{1}{2}\|\mathbf{w}\|^2$ with its gradient; minimize it with L-BFGS, starting from a linear SVM.

2. On `australian`, `pima`, `wdbc`, `spambase`, compute Seeger's certificate on the Gibbs risk for 13 values of C in $[10^{-2}, 10^2]$, select the best one, and compare the test risk of the vote $\operatorname{sign}\langle \mathbf{w}, \tilde{x}\rangle$ with a linear SVM ($C = 1$). Plot the certificate as a function of C.

3. Pretend that the Gibbs risk has no closed form: estimate it with $N = 2000$ classifiers drawn from $Q_{\mathbf{w}}$ and compute the Monte Carlo certificate (Section 4.3), with $\delta/2$ for each step. How much is lost?

4. Why can C be selected by minimizing the certificate without any union bound?

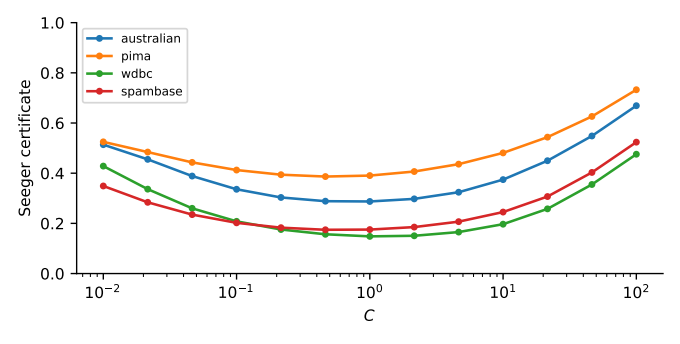
Solution

Algorithm 13: PBGD with certificate-based choice of C, and Monte Carlo certificate

for C *in a geometric grid* **do**
  $\mathbf{w}_C \leftarrow \arg\min_{\mathbf{w}} \; C\sum_i \Phi\big(-\frac{y_i\langle \mathbf{w}, \tilde{x}_i\rangle}{\|\tilde{x}_i\|}\big) + \frac{1}{2}\|\mathbf{w}\|^2$; ▷ *L-BFGS, warm start at the SVM*
  $B_C \leftarrow \overline{\mathrm{kl}}^{-1}\big(\widehat{R}_S(G_{Q_{\mathbf{w}_C}}), \frac{\|\mathbf{w}_C\|^2/2 + \ln(2\sqrt{m}/\delta)}{m}\big)$;
$\mathbf{w} \leftarrow \mathbf{w}_{C^\star}$ with $C^\star = \arg\min_C B_C$;
; ▷ *Monte Carlo version*
draw $\mathbf{v}_1, \dots, \mathbf{v}_N \sim \mathcal{N}(\mathbf{w}, I)$; $\hat{r}_N \leftarrow \frac{1}{Nm}\sum_{j,i} \mathbb{1}\big[h_{\mathbf{v}_j}(x_i) \neq y_i\big]$;
$\bar{r} \leftarrow \overline{\mathrm{kl}}^{-1}\big(\hat{r}_N, \ln(4/\delta)/N\big)$; certificate $\leftarrow \overline{\mathrm{kl}}^{-1}\big(\bar{r}, \frac{\|\mathbf{w}\|^2/2 + \ln(4\sqrt{m}/\delta)}{m}\big)$;

Python: function `probit_risk, pbgd, mc_certificate, partD` of `tp_solutions.py`.

dataset	SVM test	PBGD (best C) $C^\star$	$\|\mathbf{w}\|$	$\widehat{R}_S(G)$	Seeger cert.	MC cert.	test vote	test Gibbs
australian	0.150	1.00	5.68	0.155	0.286	0.318	0.140	0.164
pima	0.229	0.46	4.11	0.266	0.379	0.413	0.242	0.270
wdbc	0.047	1.00	5.06	0.052	0.150	0.176	0.041	0.073
spambase	0.075	0.46	10.10	0.110	0.180	0.206	0.078	0.113



2. The certificate is a U-shaped function of C (figure): for small C the posterior stays close to the prior and its Gibbs risk is near $\frac{1}{2}$; for large C the norm, hence the KL, explodes. The selected $C^\star$ is between 0.5 and 1 (average over the splits). The vote of PBGD is as accurate as the linear SVM (0.140 vs 0.150 on `australian`, 0.041 vs 0.047 on `wdbc`, slightly worse on `pima`), and it comes with a certificate: 0.150 on `wdbc` and 0.180 on `spambase`. The certificate bounds the risk of the Gibbs classifier (test value 0.073 and 0.113); the risk of the vote is at most twice this value, and is actually smaller.
3. With $N = 2000$ samples, the Monte Carlo certificate is only 0.02–0.035 larger (0.176 instead of 0.150 on `wdbc`): the price of not knowing the Gibbs risk exactly is small, which is what makes PAC-Bayesian certificates for neural networks possible.
4. C only indexes posteriors $Q_{\mathbf{w}_C}$, all compared with the same prior in the same bound (Seeger's): the certificate is valid simultaneously for all of them (Theorem 6.2). This is a real advantage over the SVM, whose C must be chosen by cross-validation.

Part 4.2 – (⋆) Data-dependent priors for linear classifiers (Sections 4.2 and 6.4)

1. Split the training set into S_1 (a fraction f) and S_2. Learn a prior centre $\mathbf{w}_0$ on S_1 (PBGD with $C = 1$), then run PBGD on S_2 with the prior $\mathcal{N}(\mathbf{w}_0, I)$, i.e. with the regularizer $\frac{1}{2}\|\mathbf{w} - \mathbf{w}_0\|^2$, and compute Seeger's certificate on S_2 only ($\mathrm{KL} = \frac{1}{2}\|\mathbf{w} - \mathbf{w}_0\|^2$), selecting C on the grid of Part 4.1.
2. Compare $f \in \{0, 0.2, 0.5\}$ ($f = 0$ is Part 4.1) on the four datasets: size of S_2, KL, certificate, test risk.
3. Which examples may the posterior use? Why is the certificate valid although $\mathbf{w}_0$ depends on the data?

Solution

Algorithm 14: Data-dependent Gaussian prior

split the training set into S_1 (f) and S_2 ($1 - f$), $m_2 = |S_2|$;
$\mathbf{w}_0 \leftarrow$ PBGD on S_1 (prior $\mathcal{N}(0, I)$, $C = 1$) ; ▷ $\mathbf{w}_0 = 0$ *if* $f = 0$
for C *in the grid* **do**
  $\mathbf{w}_C \leftarrow \arg\min_{\mathbf{w}} C \sum_{i \in S_2} \Phi\big(-\frac{y_i \langle \mathbf{w}, \tilde{x}_i \rangle}{\|\tilde{x}_i\|}\big) + \frac{1}{2}\|\mathbf{w} - \mathbf{w}_0\|^2$;
  $B_C \leftarrow \overline{\mathrm{kl}}^{-1}\big(\widehat{R}_{S_2}(G_{Q_{\mathbf{w}_C}}), \frac{\|\mathbf{w}_C - \mathbf{w}_0\|^2/2 + \ln(2\sqrt{m_2}/\delta)}{m_2}\big)$;
return the best C and its certificate;

Python: function `partE` of `tp_solutions.py`.

dataset	f	m_2	KL	certificate	test risk (vote)
australian	0.0	483	16.11	0.286	0.140
australian	0.2	387	2.82	0.246	0.132
australian	0.5	242	1.73	0.243	0.142
pima	0.0	537	8.45	0.379	0.242
pima	0.2	430	3.33	0.351	0.235
pima	0.5	269	1.01	0.366	0.242
wdbc	0.0	398	12.81	0.150	0.041
wdbc	0.2	319	4.58	0.119	0.039
wdbc	0.5	199	1.83	0.123	0.041
spambase	0.0	3217	51.03	0.180	0.078
spambase	0.2	2574	12.59	0.139	0.079
spambase	0.5	1609	4.55	0.134	0.078

2. Learning the centre of the prior on a fifth of the data divides the KL by 3 to 6 (16.1 → 2.8 on `australian`, 51.0 → 12.6 on `spambase`) and improves every certificate, although it is computed on fewer examples: 0.286 → 0.246 on `australian`, 0.150 → 0.119 on `wdbc`, 0.180 → 0.139 on `spambase`. Going to $f = 0.5$ helps further on the largest dataset (`spambase`: 0.134) but not on the small ones, where S_2 becomes too small (`pima`: 0.351 → 0.366): the same trade-off as in Figure 28 of the notes. The test risk of the vote is essentially unchanged.
3. In the version implemented here the posterior only uses S_2, but it could use all the data: only the *empirical term* of the bound has to be computed on S_2 (Proposition 6.5). The certificate is valid because, conditionally on S_1, the prior $\mathcal{N}(\mathbf{w}_0, I)$ is fixed and S_2 is an independent i.i.d. sample: the PAC-Bayesian theorem applies conditionally, and integrating over S_1 gives the result. The fraction f must be fixed in advance (or chosen on a grid with a union bound).

Part 4.3 – Synthesis: which certificate for which situation?

Using your results of the four sessions (and not new computations), fill in the following decision table. For each situation, say which guarantee you would report (test-set bound, first-order, tandem or C-bound certificate, Seeger's bound on a Gibbs classifier, with or without data-dependent prior), and justify your choice in one sentence with a number from your experiments.

situation	guarantee to report, and why
a forest has been learned, and 30% of the data can be set aside	
a small dataset (a few hundred examples)	
the weights of the vote are learned from the data	
several models are compared and one is selected	
an imbalanced problem	
a boosted vote of stumps	
a stochastic (Gibbs) linear classifier	

Solution

situation	guarantee to report, and why
fixed forest, 30% of the data available	the *test-set bound* of the vote: 3 to 5 times tighter than any self-bounding certificate (Part 3.2: 0.071 vs 0.274 on `spambase`) for almost the same vote.
small dataset	the *first-order* OOB certificate, estimated on the largest sample (n_1) and multiplied by 2 only (Part 1.3); but expect it to be loose, and prefer a hold-out certificate if the vote matters more than the bound.
weights learned from the data	the *tandem* certificate of the *TND-optimized* posterior: it is valid for a data-dependent posterior and learning with it preserves the vote (Part 2.1); never learn with the first-order bound.
model selection	certificates computed with δ/K and the smallest one, *if they are non-vacuous* (Part 2.4); otherwise cross-validation followed by a hold-out certificate.
imbalanced problem	a certificate on the *balanced error*, class by class with a union bound (Part 3.3); the 0-1 certificate cannot even beat the trivial classifier.
boosted vote of stumps	a *hold-out* certificate of the vote (Part 3.4): the Gibbs risk of weak learners makes every Gibbs-based certificate vacuous.
stochastic linear classifier	*Seeger's bound* on the Gibbs risk (exact probit risk, or Monte Carlo with a second kl inversion), with a *data-dependent prior* on a fifth of the data (Parts 4.1 and 4.2: 0.119 on `wdbc`).

Part 4.4 – Report (to finish after the session)

Summarize your results in a short report (3–4 pages) containing the tables of Parts 1.3 and 2.1, two figures of your choice and a discussion. As in the first practical session, compare methods that are comparable and give standard deviations; remember that a certificate is not an estimate of the risk, and comment on its tightness, not only on its value; finally, report every choice made using the data (grids, splits, selected bounds), since each of them may require a union bound.

Solution

A good report states the four lessons of the session. First, certificates are only informative when the voters are evaluated on enough examples (a few thousand OOB examples for a random forest): on small datasets they are vacuous, and this is not a failure of the implementation. Second, the choice of the bound matters twice: as a certificate, the first-order bound is often the tightest on small and medium datasets and the tandem bound on large ones; as a learning objective, only the second-order bound preserves the diversity and the accuracy of the vote. Third, for a fixed vote the hold-out certificate is much tighter, while self-bounding certificates shine when they are used to learn, to select, or to certify stochastic predictors, especially with data-dependent priors. Fourth, every data-dependent choice has a price, usually small ($\ln K$), but it must be paid.